

Signals Deep Dive

フロントエンド・PHPカンファレンス北海道 2026

nishitaku

初めまして



西濃 拓郎（にしの たくろう）

[@nishitaku](https://twitter.com/nishitaku)

フリーランス（10年目）



Signals 知ってる人～ 🙌

これから話すこと

Signals がどういう仕組みで
どうやって必要な箇所だけを更新しているのか

持ち帰ってもらいたいこと

Signals の内部実装を理解して
リアクティブシステムの勘所を掴む

目次

1. Signals 概要

2. Signals の特徴

特徴① 依存関係の自動追跡

特徴② Push-Pull ハイブリッド方式

特徴③ メモ化

3. Signal Polyfill の実装を読む

Signals 概要

Signals 概要

```
const counter = new Signal.State(0);  
const isEven = new Signal.Computed(() => (counter.get() & 1) == 0);  
effect(() => { element.innerText = isEven.get() });
```

- 状態と依存関係を効率的に扱うリアクティブモデル
- 各種フレームワーク(Angular/Preact/Svelte/Vue ...etc)で採用
- 基本3要素
 - **State**: 手動で設定される値
 - **Computed**: Stateに依存して計算される値
 - **Effect**: StateやComputedに依存して実行されるコールバック
- TC39 proposal-signals (Stage 1)

Signals の特徴①

依存関係の自動追跡

Vanilla JSの場合

```
let counter = 0;
const setCounter = (value) => {
  counter = value;
  render();
};

const isEven = () => (counter & 1) == 0;
const parity = () => isEven() ? "even" : "odd";
const render = () => element.innerText = parity();

setInterval(() => setCounter(counter + 1), 1000);
```

開発者が依存関係を知っておく必要がある

→ 変更時の対応漏れや過剰更新が問題になり、スケールしない

Signals の場合

```
const counter = new Signal.State(0);  
const isEven = new Signal.Computed(() => (counter.get() & 1) == 0);  
const parity = new Signal.Computed(() => isEven.get() ? "even" : "odd");  
  
effect(() => {  
  element.innerText = parity.get();  
});  
  
setInterval(() => counter.set(counter.get() + 1), 1000);
```

依存関係を自動で追跡してくれる

→ 依存関係が増えてもスケールしやすい

宣言的UIの場合

```
// JavaScript
let name = "太郎";
let age = 20;
const validatedName = () => f(name); // nameに依存
const parity = () => (age % 2 === 0 ? '偶数' : '奇数'); // ageに依存

// HTML
<p>名前は{{ validatedName() }}です</p>
<p>年齢は{{ parity() }}です。</p>
```

依存関係がわからないと、**age** が変更されたときに
parity だけでなく **validatedName** も再計算する必要がある

Signals の場合

```
// JavaScript
const name = signal("太郎");
const age = signal(20);
const validatedName = computed(() => f(name()));
const parity = () => (age() % 2 === 0 ? '偶数' : '奇数');

// HTML
<p>名前は{{ validatedName() }}です</p>
<p>年齢は{{ parity() }}です。</p>
```

効率的に必要な計算だけを再実行できる

**Signals を使うことで
自動で依存関係を追跡できる**

Signals の特徴②

Push-Pull ハイブリッド方式

データを誰が主導して渡すか

Push型：データを持っている側が通知する

- 例) Pub/Sub、WebSocket、Observerパターン、EventEmitter
- 常に最新状態
- 無駄な再計算が発生しやすい

Pull型：データを使う側が取りに行く

- 例) ポーリング、getter、関数呼び出し
- 必要なときに取得して計算
- 毎回取得コストが発生してしまう

Signals は Push-Pull ハイブリッド方式

```
const counter = new Signal.State(0);  
const isEven = new Signal.Computed(() => (counter.get() & 1) == 0);
```

Stateの変更は即座に通知される**Push型**



Computedの評価は**Pull型**

Signals の特徴③

メモ化

Computed のメモ化

```
const parity = () => (age() % 2 === 0 ? '偶数' : '奇数');
```

依存先のdirtyフラグをキャッシュキーとしている

- `age` のdirtyフラグが `true` → 再計算した値を返す
- `age` のdirtyフラグが `false` → キャッシュ値を返す

Signal Polyfill の実装を読む

Signal Polyfill とは

TC39 proposal-signals のポリフィル実装

```
import { Signal } from "signal-polyfill";
import { effect } from "./effect.js";

const counter = new Signal.State(0);
const isEven = new Signal.Computed(() => (counter.get() & 1) == 0);
const parity = new Signal.Computed(() => (isEven.get() ? "even" : "odd"));

effect(() => console.log(parity.get())); // Console logs "even" immediately.
setInterval(() => counter.set(counter.get() + 1), 1000); // Changes the counter every 1000ms.

// effect triggers console log "odd"
// effect triggers console log "even"
// effect triggers console log "odd"
// ...
```

依存関係の追跡を実現するための3ステップ

依存グラフの構築

読むことで依存を登録する

Push

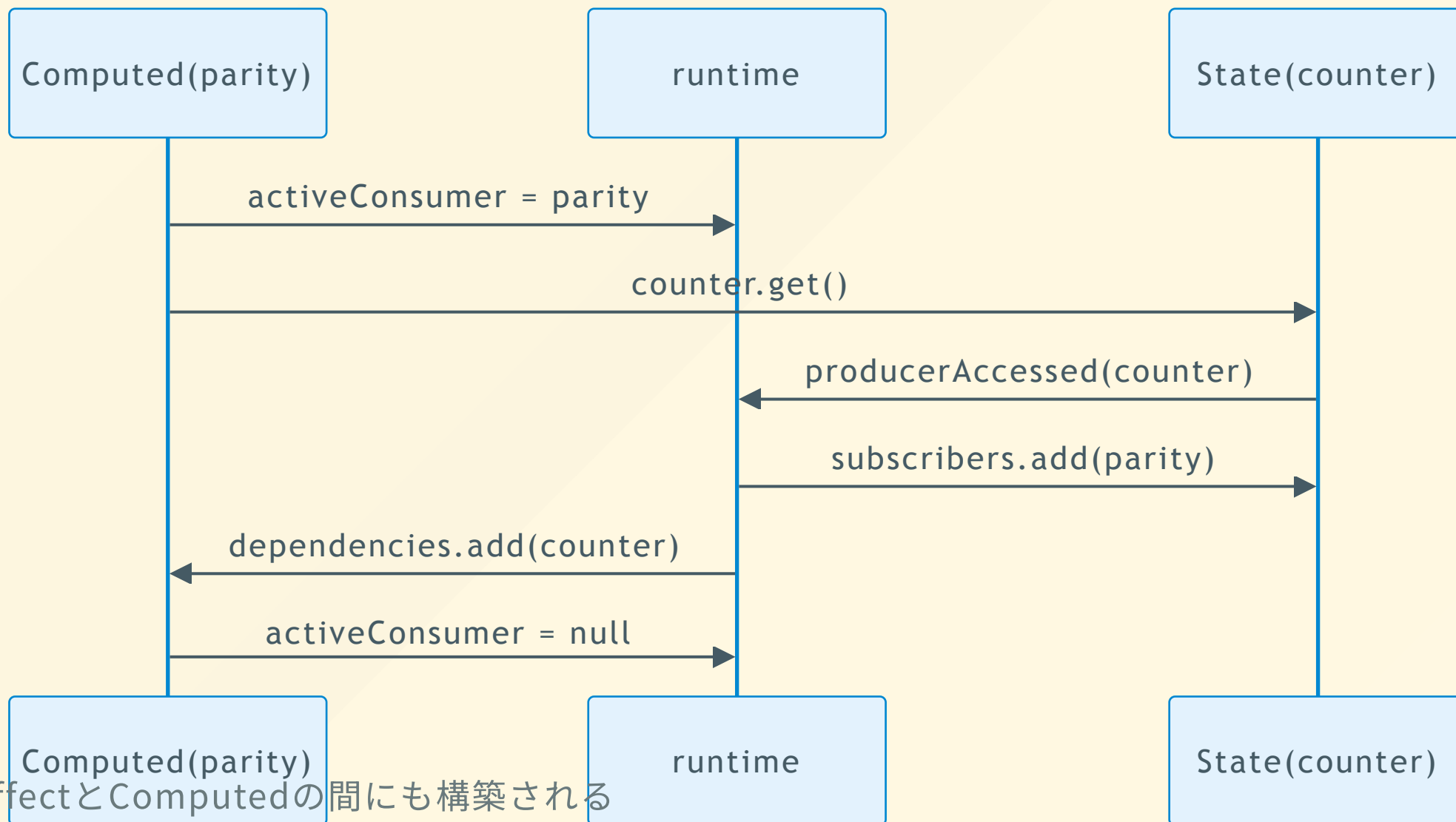
「古くなった」を伝播する

Pull

必要になったときだけ再計算する

依存グラフの構築

```
const counter = new Signal.State(0);  
  
const parity = new Signal.Computed(() => (counter.get() % 2) == 0 ? "even" : "odd");  
  
effect(() => { element.innerText = parity.get() });
```

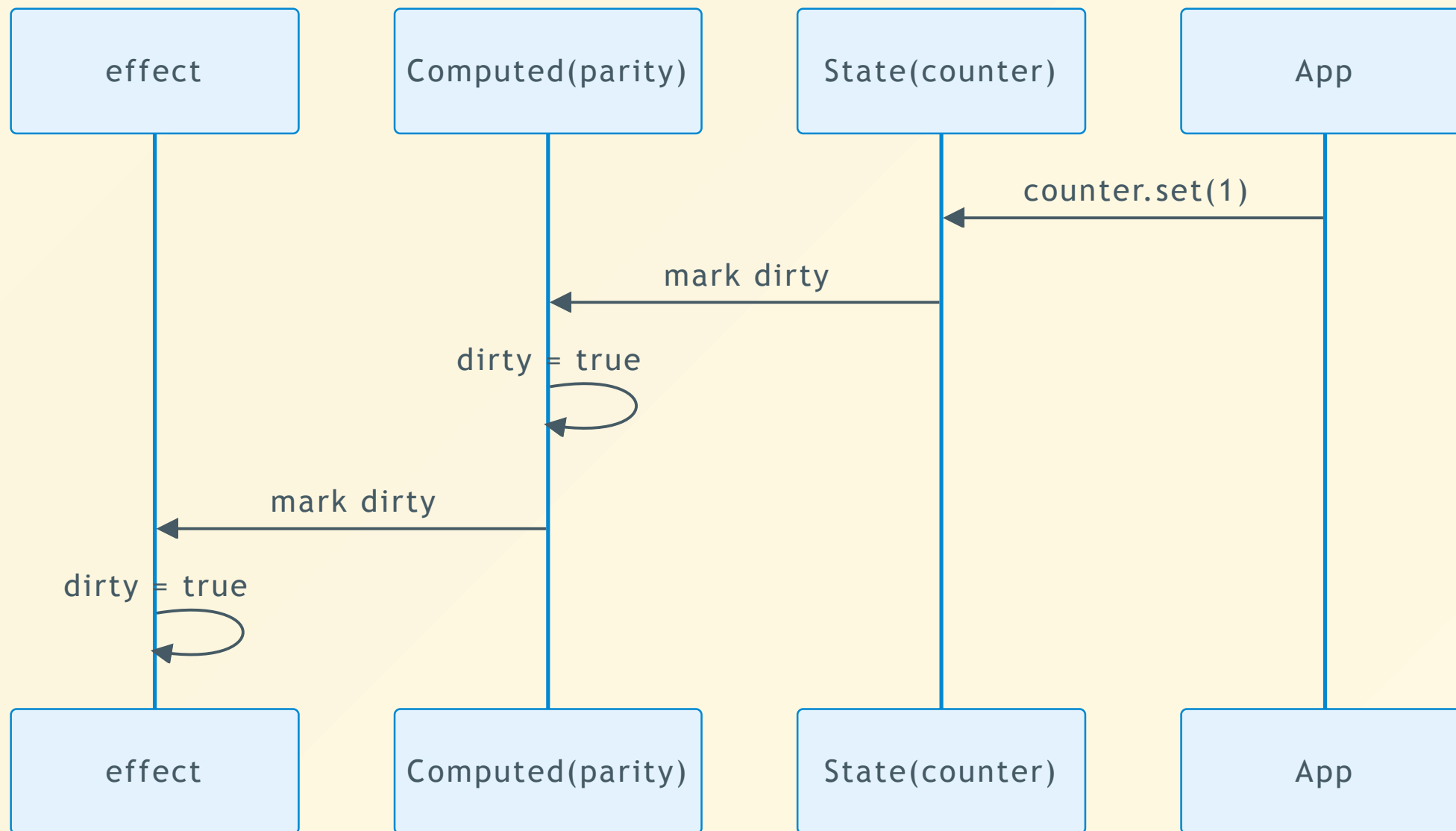


Pushフェーズ

```
const counter = new Signal.State(0);
```

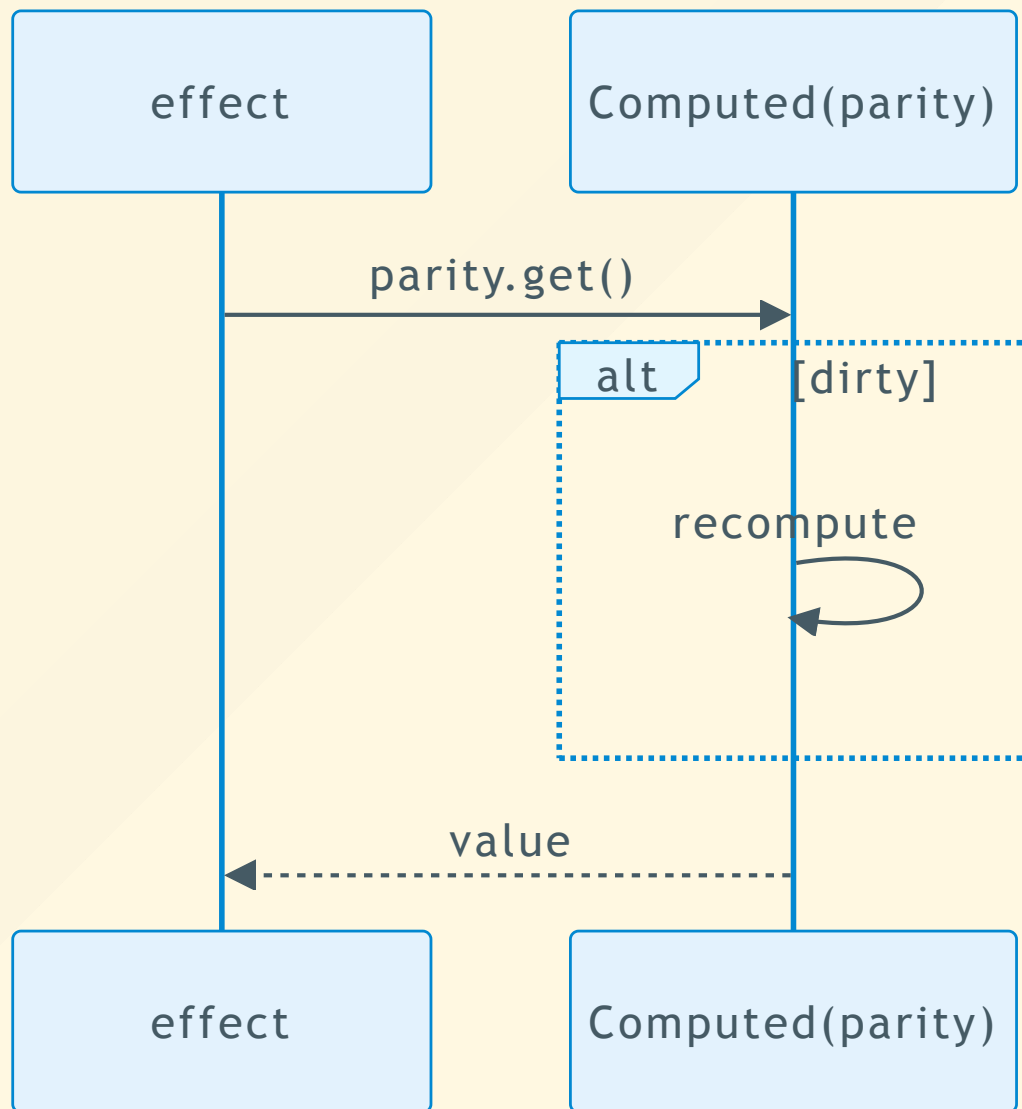
```
const parity = new Signal.Computed(() => (counter.get() % 2) == 0 ? "even" : "odd");
```

```
effect(() => { element.innerText = parity.get() });
```



Pullフェーズ

```
const counter = new Signal.State(0);  
const parity = new Signal.Computed(() => (counter.get() % 2) == 0 ? "even" : "odd");  
effect(() => { element.innerText = parity.get() });
```



Signal Polyfill の実装

- 依存関係の追跡は、双方向の依存グラフによって実現
 - データを保持する側 `producer`
 - `producer.subscribers` に `consumer` を登録
 - データを取得する側 `consumer`
 - `consumer.dependencies` に `producer` を登録

Signals Deep Dive のまとめ

- Signals は「誰が誰を読んだか」の依存グラフを構築する
- 依存グラフにより変更伝搬を自動化する
- Push-Pull ハイブリッド方式により必要時だけ再計算する

ご清聴ありがとうございました

nishitaku

Why Signals Deep Dive ?

- Angular と zone.js
- v16 で 導入された Signals によって環境が激変
- Signals の魔法に感動

Runtime Dependency Tracking

- ビルド時ではなく、実行時に依存を解決している

```
const value = computed(() => {  
  if (flag()) {  
    return count()  
  }  
  
  return another()  
})
```

グリッチフリー

```
count = 1  
count = 2  
count = 3
```

- グリッチ：中間の不正確な値が表示されること
- Push型リアクティブモデルの場合、変更が即座にUIに反映される
- Signals はフレームワークが UI を描画するタイミングで必要な更新だけを取りに行くため、グリッチが発生しづらい
- 「損失がある」ことの裏返しでもある。

Signals のトレードオフ

- 「誰が誰を読むか」が実行時に決まる
- 更新の流れを追うのが難しいことがある
- 細かく分割しすぎると複雑になりやすい
- Signals を使わない方がシンプルな場合もある

Signals は observer pattern ?

- 広義のobserver pattern を含んでいる
- しかし本質は dependency graph
- Signalsは「読むことで依存登録」が重要
- computed による派生状態を扱える
- Push/Pull Hybrid によって lazy に再計算する

TC39 proposal-signals Stage 1

- フレームワーク共通のリアクティブモデルを目指している
- いくつかの課題をクリアしてから Stage 2 へ
 - 複数の production-grade polyfill
 - 複数 framework への統合検証
 - パフォーマンス検証

React vs Signals

- 設計思想が異なる
- [Async React の設計思想と Signal の違いを Transition を中心に考える](#)
- [React vs Signals: 10 Years Later](#)

References

TC39 Signals

- [tc39/proposal-signals](#)
- [signal-polyfill](#)
- [A TC39 Proposal for Signals](#)